BTS SIO – Projet Spécial E4

Django – 10/10/2025

Esicad – Camille GOSSET - 2025

Introduction à Django

- Framework web Python (créé en 2005)
- Utilisé pour développer des applications web rapidement
- Philosophie : "Don’t Repeat Yourself" (DRY)
- Sécurité, rapidité, productivité

Pourquoi Django ?

- Complet et « batteries incluses »
- Sécurisé (protège contre XSS, CSRF, injections SQL…)
- Architecture robuste (MVC / MTV)
- Utilisé par : Instagram, Spotify, YouTube (parties), etc.

Architecture MTV

- **Model :** gère la base de données
- **Template :** gère l’affichage (HTML)
- **View :** gère la logique de l’application
- **URL dispatcher :** relie les requêtes HTTP aux vues

Première étape : Installation

1. Vérifier Python (≥3.10 recommandé)
2. Installer Django `pip install django`
3. Créer un projet `django-admin startproject monprojet`
4. Lancer le serveur

```
python manage.py runserver
```

Structure d’un projet Django

- manage.py : outils de gestion
- Dossier du projet : paramètres globaux
- Applications (apps) : modules réutilisables
- Paramètres (settings.py), URLs (urls.py)

Créer une application

1. Ajouter une app
2. Fichiers générés : `models.py`, `views.py`, `urls.py` (souvent ajouté)
3. Déclarer l’app dans `settings.py` (INSTALLED_APPS)

Créer une application

1. Ajouter une app
2. Fichiers générés : `models.py`, `views.py`, `urls.py` (souvent ajouté)
3. Déclarer l’app dans `settings.py` (INSTALLED_APPS)

Exemple simple : Hello World

View.py :

```
from django.http import HttpResponse

def hello(request):
    return HttpResponse("Hello, world!")
```

URL.py :

```
from django.urls import path
from . import views

urlpatterns = [
    path("hello/", views.hello),
]
```

Exemple simple : Hello World

View.py :

```python
from django.http import HttpResponse

def hello(request):
    return HttpResponse("Hello, world!")
```

URL.py → Routeur de l'application Django

URL.py :

```python
from django.urls import path
from . import views

urlpatterns = [
    path("hello/", views.hello),
]
```

Exemple simple : Hello World

1. **Import de la fonction path : définir une route**
    1. Route = Correspondance entre une **URL** et une **vue**
2. **Importer le module views du même dossier (fonctions de vue)**
3. **Règle de routage :**
    1. "hello/" = le chemin d'URL à capturer
    2. views.hello = la fonction Python à exécuter quand quelqu'un visite cette URL

URL.py :

```
from django.urls import path
from . import views

urlpatterns = [
    path("hello/", views.hello),
]
```

Base de données et Models

1. Définir un modèle (classe python)

```
from django.db import models

class Article(models.Model):
    titre = models.CharField(max_length=100)
    contenu = models.TextField()
    date_pub = models.DateTimeField(auto_now_add=True)
```

2. Migration

```
python manage.py makemigrations
python manage.py migrate
```

Base de données et Models

1. Définir un modèle (classe python)

```python
from django.db import models

class Article(models.Model):
    titre = models.CharField(max_length=100)
    contenu = models.TextField()
    date_pub = models.DateTimeField(auto_now_add=True)
```

1. Module *models* fourni par Django
   Utiliser les types de champs
2. Classe Article : hérite de models.Model → créer une table SQL
3. Créer les différentes colonnes

Base de données et Models

1. Définir un modèle (classe python)

```python
from django.db import models

class Article(models.Model):
    titre = models.CharField(max_length=100)
    contenu = models.TextField()
    date_pub = models.DateTimeField(auto_now_add=True)
```

Champ	Type Django	Type SQL équivalent	Exemple
titre	CharField (100)	VARCHAR (100)	"Mon premier article"
contenu	TextField	TEXT	"Voici le contenu de l'article..."
date_pub	DateTimeField	DATETIME	"2025-10-10 09:32:00"

Base de données et Models

2. Migration

```
python manage.py makemigrations
python manage.py migrate
```

1. Traduction SQL → Python
2. Execution du SQL dans la Base de données

Manage.py

- **Outil central → Télécommande 📠 du projet Django**
- **Script de gestion: communiquer avec Django**

Commande	Rôle
`python manage.py runserver`	Lance le serveur web local
`python manage.py startapp nom_app`	Crée une nouvelle application
`python manage.py makemigrations`	Prépare les migrations
`python manage.py migrate`	Applique les migrations à la BDD
`python manage.py createsuperuser`	Crée un compte admin
`python manage.py shell`	Ouvre la console Python avec ton projet chargé
`python manage.py dumpdata`	Exporte le contenu de la base
`python manage.py loaddata`	Importe des données

Comment renseigner ma BDD?

Étape	Action	Exemple
1	Ouvrir settings.py	monprojet/monprojet/settings.py
2	Modifier la section DATABASES	Choisir sqlite3, mysql ou postgresql
3	Installer le connecteur si nécessaire	pip install mysqlclient
4	Lancer les migrations	python manage.py migrate

Fichier « settings.py »

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Bases de données possibles

Type de base	Valeur de 'ENGINE'	Exemple de 'NAME' et paramètres
SQLite (par défaut)	'django.db.backends.sqlite3'	'NAME': BASE_DIR / 'db.sqlite3'
PostgreSQL	'django.db.backends.postgresql'	python 'NAME': 'ma_base', 'USER': 'postgres', 'PASSWORD': 'motdepasse', 'HOST': 'localhost', 'PORT': '5432',
MySQL	'django.db.backends.mysql'	python 'NAME': 'ma_base', 'USER': 'root', 'PASSWORD': 'motdepasse', 'HOST': '127.0.0.1', 'PORT': '3306',
Oracle	'django.db.backends.oracle'	... (moins courant en BTS 😉)

Exemple complet : PostgreSQL

1 – Settings.py

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'blogdb',
        'USER': 'monuser',
        'PASSWORD': 'monmotdepasse',
        'HOST': 'localhost',
        'PORT': '5432',
    }
}
```

2 – Installer le connecteur python

```
pip install psycopg2
```

3 – Créer les tables dans la base PostegreSQL

```
python manage.py makemigrations
python manage.py migrate
```

Exemple complet : MySQL

1 – Settings.py

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'blogdb',
        'USER': 'root',
        'PASSWORD': '1234',
        'HOST': 'localhost',
        'PORT': '3306',
    }
}
```

2 – Installer le driver MySQL

```
pip install mysqlclient
```

3 – Créer les tables dans la base

```
python manage.py makemigrations
python manage.py migrate
```